기초 파이썬

2026. 9. 16일

담당교수: 박진호

서강대학교 경제대학원

2026년 2학기

◈ 오늘의 학습 목표

▣ 수치 연산 라이브러리인 넘파이(Numpy)에 관해 학습한다.

- ■ 넘파이 배열의 개념과 설치에 대해 배운다.
- ■ 넘파이 배열을 생성해본다.
- ■ 넘파이 배열을 변환해본다.

▣ 데이터를 분석하는 라이브러리인 판다스(Pandas)에 관해 학습한다.

- ■ 판다스 설치에 대해 배운다.
- ■ Series와 DataFrame을 생성해본다.
- ■ 판다스 데이터를 조정(삭제, 결합, 그룹화)해본다.

4장 넘파이, 판다스

◈ 파이썬 라이브러리 소개: 필수적인 파이썬 라이브러리

1. 넘파이(Numpy)

□ Numpy는 Numerical Python의 준말로 파이썬 산출 계산의 주춧돌과 같은 라이브러리

□ 자료구조, 알고리즘 산출 데이터를 다루는 대부분의 과학 계산 애플리케이션에서 필요한 라이브러리를 제공

2. 판다스(pandas)

□ pandas는 다차원으로 구조화된 데이터를 의미하는 panel data와 Python data analysis에서 그 이름을 따온 것으로

구조화된 데이터 또는 표 형식의 데이터를 직관적이고 유연하게 다룰 수 있도록 설계된 고수준의 자료구조와 함수를 제공

□ pandas의 주요 자료 구조는 표 형태의 행과 열을 갖는 DataFrame과 1차원 배열 객체인 Series가 있음

3. 맷플롯립(Matplotlib)

□ Matplotlib는 도표나 2차원 데이터 시각화를 생성하는 파이썬 라이브러리로서 출판물에 적합한 도표를 생성하도록 설계됨

4. 사이파이(SciPy)

□ SciPy는 과학 계산 영역의 다양한 문제를 다루는 패키지 모음

5. 사이킷런(scikit-learn)

□ scikit-learn은 파이썬 개발자를 위한 범용 머신러닝 도구로서 인기를 끌고 있으며 아래와 같은 모델을 위한 하위 모듈(submodule)을 포함

- o 분류: SVM, 최근접 이웃, 랜덤 포레스트, 로지스틱 회귀 등
- o 회귀: Lasso, ridge regression 등

- 클러스터링: k-평균, 스펙트럼 클러스터링 등
- 차원 축소: PCA, 특징 선택, 행렬 인수분해 등
- 모델 선택: 격자 탐색, 교차 검증, 행렬
- 전처리: 특징 추출, 정규화

⇒ pandas, statsmodels, scikit-learn은 파이썬이 생산적인 데이터 과학 언어로 자리잡는데 큰 역할을 함

6. statsmodels

□ statsmodels는 다양한 R 언어용 회귀분석 모델에 기반한 통계분석 패키지

□ 자료구조, 알고리즘 산출 데이터를 다루는 대부분의 과학 계산 애플리케이션에서 필요한 라이브러리를 제공

4.1 넘파이(numpy)

4.1 NumPy를 이용한 배열과 수치 계산

4.1.1 NumPy란?

□ 데이터를 분석할 때는 여러 개의 숫자를 동시에 처리해야 하는 경우가 많음

- 예) 여러 기업의 매출액 비교, 여러 기간의 경제성장률 계산, 많은 관측치의 평균 계산

□ NumPy(Numerical Python)는 이러한 수치 계산을 효율적으로 수행하기 위해 사용하는 Python의 대표적인 라이브러리

- NumPy의 핵심은 여러 개의 숫자를 하나로 묶어서 처리할 수 있는 배열(array)임
- NumPy는 이후에 배우게 될 pandas 등 여러 데이터 분석 라이브러리의 기반이 되기도 함

□ NumPy는 일반적으로 다음과 같이 불러옴. ‘np‘는 NumPy를 간단히 사용하기 위해 관례적으로 사용

```
import numpy as np
```

4.1.2 List와 Array

(1) List를 이미 알고 있다면 Array는 어렵지 않음

□ 앞에서 우리는 여러 개의 값을 하나의 변수에 저장하기 위해 리스트(list)를 사용하였음

▸ 예) 세 기업의 매출액이 각각 100억 원, 120억 원, 150억 원이라면 다음과 같이 저장 가능

```
sales_list = [100, 120, 150]
print(sales_list)
```

결과: [100, 120, 150]

□ NumPy에서도 여러 값을 하나로 묶어 저장할 수 있다. 이를 배열(array)이라고 함

```
import numpy as np
sales_array = np.array([100, 120, 150])
```

```
print(sales_array)
```

결과: [100 120 150]

□ 두 결과는 겉으로 보기에는 매우 비슷한데, 그렇다면 왜 굳이 Array를 사용할까?

(2) Array는 여러 숫자를 계산하기 편함

□ 리스트와 배열의 가장 중요한 차이는 "숫자를 계산하는 방법"에서 확인 가능

□ 세 기업의 매출액을 2배로 증가시켜 보면,

```
sales_list = [100, 120, 150]

sales_list * 2
```

결과: [100, 120, 150, 100, 120, 150]

▸ 리스트의 각 값이 2배가 된 것이 아니라 "리스트 자체가 두 번 반복됨"

□ 이번에는 NumPy 배열을 사용해 보면,

```
sales_array = np.array( [100, 120, 150] )

sales_array * 2
```

결과: array([200, 240, 300])

▸ 이번에는 배열에 들어 있는 모든 숫자에 2가 곱해졌음

□ 따라서 정리하면,

“List는 여러 값을 저장하고 관리하는 데에, Array는 여러 숫자를 한꺼번에 계산하는 데 편리”

4.1.3 1차원과 2차원 배열

□ NumPy 배열은 한 줄로 된 자료뿐 아니라 행과 열로 구성된 표 형태의 자료도 저장 가능

(1) 1차원 배열

□ 한 변수에 대한 여러 개의 관측값을 한 줄로 저장하면 1차원 배열이 됨

```
sales = np.array([100, 120, 150, 180])
print(sales)
```

결과: [100 120 150 180]

□ 예) 4개 기업의 매출액, 1개 기업의 4년간 매출액 등을 나타낼 수 있음

(2) 2차원 배열

□ 여러 개의 행과 열을 가진 자료는 2차원 배열로 나타낼 수 있음

□ 예) 3개 기업의 매출액과 영업이익 자료가 다음과 같음

기업	매출액	영업이익
A	100	10
B	120	15
C	150	20

□ 이를 NumPy 배열로 만들면,

```
company = np.array([ [100, 10], [120, 15], [150, 20] ])

print(company)
```

결과:

```
[[100   10]
 [120   15]
 [150   20]]
```

▸ 여기서는 각 행이 하나의 기업, 각 열이 하나의 변수라고 생각하면 이해하기 쉽다.

4.1.4 배열의 기본 정보 확인

□ 배열을 만든 후에는 배열의 크기와 자료형 등을 확인할 수 있음

```
company = np.array( [ [100, 10], [120, 15], [150, 20] ] )
```

(1) shape: 행과 열의 수

```
company.shape
```

▸ 결과: (3, 2) ⇦ 3개의 행과 2개의 열을 의미

(2) ndim: 차원의 수

company.ndim

- 결과: 2 ⇦ 현재 배열이 2차원이라는 의미

(3) size: 전체 원소의 수

company.size

- 결과: 6 ⇦ 3행 × 2열이므로 총 6개의 값이 있음

(4) dtype: 원소의 자료형

company.dtype

- 결과: dtype('int64') ⇦ 배열에 저장된 값의 자료형을 알려줌

4.1.5 배열 만들기

□ NumPy에서는 리스트를 np.array()로 변환하는 것 외에도 일정한 규칙을 가진 배열을 쉽게 만들 수 있음

(1) ‘zeros()‘와 ‘ones()‘

▸ 모든 값이 0인 배열:

```
np.zeros(5)
```

```
array([0., 0., 0., 0., 0.])
```

▸ 모든 값이 1인 배열:

```
np.ones(5)
```

```
array([1., 1., 1., 1., 1.])
```

(2) arange()

◇ range()와 마찬가지로 시작값은 포함하고 마지막 값은 포함하지 않음

- 일정한 간격을 가진 숫자를 만들 때 사용한다.

```
np.arange(1, 11)
```

- 결과: array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

- 간격도 지정 가능

```
np.arange(0, 11, 2)
```

- 결과: array([0, 2, 4, 6, 8, 10])

(3) linspace()

◇ 시작값과 마지막 값을 지정한 개수만큼 일정한 간격으로 나눔

np.linspace(0, 100, 5)

- 결과: array([0., 25., 50., 75., 100.])

◇ arange()와 linspace()는 기준이 다름

- arange() → 간격(step)을 지정
- linspace() → 개수(number)를 지정

4.1.6 배열에서 필요한 값 선택하기

□ 배열에서도 리스트와 마찬가지로 인덱싱(indexing)과 슬라이싱(slicing) 사용이 가능

(1) 인덱싱

```
sales = np.array([100, 120, 150, 180])
```

- 첫 번째 값을 선택하면

```
sales[0]
```

- 결과: 100

- Python의 인덱스는 0부터 시작

(2) 슬라이싱

sales[1:3]

- 결과: array([120, 150])
- 1:3은 인덱스 1부터 시작하여 인덱스 3 바로 앞까지 선택

  즉, 앞은 포함하고 뒤는 포함하지 않는다.

(3) 2차원 배열

company = np.array([[100, 10], [120, 15], [150, 20]])

- 두 번째 기업의 매출액을 선택하려면

company[1, 0]

- 결과: 120
- [1, 0]에서 첫 번째 숫자는 행, 두 번째 숫자는 열의 위치를 표시

4.1.7 조건에 맞는 값 선택하기

□ NumPy에서는 조건을 이용하여 필요한 값만 선택할 수도 있음 → Boolean indexing이라 함

```
sales = np.array([100, 120, 80, 150, 90])
```

□ 100보다 큰 매출액만 선택

```
sales[sales > 100]
```

- 결과: array([120, 150])
- 먼저 sales > 100 을 실행하면 각 값이 조건을 만족하는지를 'True'와 'False'로 판단
- 이러한 조건을 배열에 적용하면 조건을 만족하는 값만 추출됨.

4.1.8 배열의 연산

□ NumPy 배열의 주요 장점은 배열의 모든 값에 동일한 계산을 한꺼번에 적용할 수 있다는 것

```
sales = np.array([100, 120, 150])
```

(1) 스칼라와의 연산

□ 모든 매출액이 10% 증가한다고 가정하면

```
sales * 1.1
```

▸ 결과: array([110., 132., 165.])

□ 모든 값에 10을 더할 수도 있음

```
sales + 10
```

▸ 결과: array([110, 130, 160])

(2) 배열 간 연산

□ 두 해의 매출액 차이는

```
sales_2025 = np.array([100, 120, 150])
sales_2026 = np.array([110, 150, 165])

sales_2026 - sales_2025
```

▸ 결과: array([10, 30, 15])

□ 따라서 각 기업의 매출증가율도 한 번에 계산할 수 있음

```
growth = (sales_2026 - sales_2025) / sales_2025 * 100

print(growth)
```

▸ 결과: [10. 25. 10.]

4.1.9 배열의 요약 통계량

□ 여러 숫자로 이루어진 배열의 합계, 평균, 최댓값 등을 간단하게 계산할 수 있음

```
sales = np.array([100, 120, 150, 180])


np.sum(sales)          # 합계
np.mean(sales)         # 평균
np.max(sales)          # 최댓값
np.min(sales)          # 최솟값
np.std(sales)          # 표준편차
np.median(sales)       # 중앙값
```

4.1.10 axis: 행과 열 방향의 계산

□ 2차원 배열에서는 전체 값의 평균뿐 아니라 행 또는 열별로 평균을 계산하기도 함

□ 다음은 3개 기업의 2025년과 2026년 매출액은

sales = np.array([[100, 120], [150, 180], [200, 220]])

기업	2025	2026
A	100	120
B	150	180
C	200	220

□ 전체 평균은: np.mean(sales)

□ 연도별 평균은: np.mean(sales, axis=0)

▸ 결과: array([150. , 173.33333333])

□ 기업별 두 해의 평균은: np.mean(sales, axis=1)

▸ 결과: array([110., 165., 210.])

◆ axis=0 : 행 방향을 따라 내려가면서 계산한다.

따라서 각 행의 값들이 계산에 사용되고 행 차원은 사라지며, 각 열에 대한 결과가 남는다

즉, $\frac{100+150+200}{3}=150$

◆ axis=1 : 두 번째 축인 열 방향을 따라 계산한다.

따라서 열 차원이 사라지고, 각 행에 대한 결과가 남는다.

즉, $\frac{100+120}{2}=110,\ \frac{150+180}{2}=165,\ \frac{200+220}{2}=210$

4.1.11 배열의 형태 바꾸기: reshape()

□ reshape()는 배열에 들어 있는 값은 그대로 두고 배열의 모양(shape)을 변경

```
x = np.arange(1, 7)
print(x)
```

▸ 결과: [1 2 3 4 5 6]

▸ 이를 2행 3열로 바꾸면

```
x.reshape(2, 3)
```

▸ 결과: [[1 2 3]
[4 5 6]]

▸ reshape()를 사용할 때는 변경 전과 변경 후의 전체 원소 수가 같아야 함

▸ 예) 6개의 원소를 가진 배열을 (2, 3) 또는 (3, 2)로 바꿀 수 있지만 (2, 4)로 바꿀 수는 없음

4.1.12 배열에 값 추가 · 삽입 · 삭제하기

(1) append(): 새로운 값을 배열의 뒤에 추가

```
sales = np.array([100, 120, 150])
sales2 = np.append(sales, 180)
print(sales2)
```

▸ 결과: [100 120 150 180]

(2) insert(): 원하는 위치에 값을 삽입

```
sales = np.array([100, 120, 150])
sales2 = np.insert(sales, 1, 110)
print(sales2)
```

▸ 결과: [100 110 120 150]

▸ 1은 새로운 값을 삽입할 인덱스 위치

(3) delete(): 원하는 위치에 있는 값을 삭제

```
sales = np.array([100, 120, 150])
sales2 = np.delete(sales, 1)
print(sales2)
```

▸ 결과: [100 150]

4.1.13 여러 배열을 하나로 결합하기

(1) concatenate(): 배열 이어 붙이기

▸ concatenate()는 여러 배열을 기존 축을 따라 이어 붙이는 결합 함수

▸ np.concatenate()의 기본값은 axis=0

◇ axis=0: 행 방향으로 결합 → 행이 늘어난다.

▸ 각각의 행을 기업, 두 열을 매출액과 영업이익이라 하면:

```
x = np.array([ [100, 10], [120, 15] ])
y = np.array([ [150, 20] ])
np.concatenate((seoul, busan), axis=0)
```

▸ 결과:

```
array([[100,  10],
       [120,  15],
       [150,  20]])
```

◇ axis=1: 열 축을 따라 결합 → 열이 늘어난다.

```
profit_rate = np.array([ [10], [12] ])
np.concatenate((x, profit_rate), axis=1)
```

▸ 결과:

```
array([[100,   10,   10],
       [120,   15,   12]])
```

(2) vstack(): 배열을 위아래로 쌓기

▸ vstack의 v는 vertical(수직)을 의미

```
np.vstack((seoul, busan))
```

▸ 결과: array([[100, 120, 150],
[80, 90, 110]])

(3) hstack(): 배열을 좌우로 붙이기

- hstack의 h는 horizontal(수평)을 의미

```
sales = np.array([ [100], [120], [150] ])
profit = np.array([ [10], [15], [20] ])
```

- 매출액 열 옆에 영업이익 열을 붙이면

```
company = np.hstack((sales, profit))
print(company)
```

- 결과: [[100 10]
  [120 15]
  [150 20]]

4.2 Pandas를 이용한 표 형태의 데이터 처리

4.2.1 Pandas는 왜 사용하는가?

□ Pandas는 표 형태의 데이터를 편리하게 저장하고 처리하기 위해 사용하는 Python의 대표적인 데이터 분석 라이브러리

□ NumPy가 주로 수치 계산과 배열 처리에 초점을 둔다면, Pandas는 행과 열로 이루어진 데이터를 다루는 데 초점을 둠

□ Pandas는 일반적으로 다음과 같이 불러옴

```
import pandas as pd
```

▸ pd는 Pandas를 간단하게 사용하기 위한 별칭

4.2.2 Series와 DataFrame

□ Pandas에는 두 가지 중요한 자료구조가 있음

□ Series는 한 줄의 데이터를 나타내는 1차원 자료구조이고, DataFrame은 여러 개의 행과 열을 가진 2차원 자료구조

(1) Array에서 Series로

□ 앞 절에서 다음과 같은 NumPy 배열을 생각해 보자.

```
export = np.array([100, 150, 200])
```

□ 이 자료에는 세 개의 수출액이 있지만, 각각 어느 나라의 수출액인지는 알 수 없음

□ Pandas의 Series에서는 각 숫자에 이름을 붙일 수 있음

```
import pandas as pd
export = pd.Series( [100, 150, 200], index=[“S.Korea“, “USA“, “China“] )
print(export)
```

▸ 결과:

```
S.Korea      100
USA           150
China         200
dtype: int64
```

□ 왼쪽의 ‘S.Korea‘, ‘USA‘, ‘China‘를 인덱스(index)라고 함

□ 따라서 Series는 기초 단계에서 다음과 같이 이해할 수 있음

Array = 값들의 배열

Series = 각각의 값에 이름(index)을 붙인 1차원 데이터

(2) Series에서 DataFrame으로

□ 이제 2025년뿐 아니라 2026년의 수출액도 있다고 하자.

```
s_2025 = pd.Series( [100, 150, 200], index=[“S.Korea“, “USA“, “China“] )
s_2026 = pd.Series( [110, 160, 210], index=[“S.Korea“, “USA“, “China“] )
```

□ 두 Series를 하나의 표로 만들면 다음과 같은 형태가 된다.

	Exp_2025	Exp_2026
S.Korea	100	110
USA	150	160
China	200	210

◆ Pandas에서는 이렇게 행과 열로 구성된 표 형태의 자료를 DataFrame이라 함

◆ 지금까지 배운 자료구조를 정리.

- List : 여러 값을 저장. Array : 여러 숫자를 한꺼번에 계산
- Series : 값에 index를 붙임. DataFrame : 여러 변수를 행과 열의 표로 구성

4.2.3 DataFrame 만들기

□ 다음의 미국과 한국의 GDP 자료를 DataFrame으로 변환

```
df = {
    "country": ["US", "US", "US", "Korea", "Korea", "Korea"],
    "year": [2020, 2021, 2022, 2021, 2022, 2023],
    "GDP": [2.1, 2.2, 2.3, 10.2, 12.3, 15.1]
}

df_fr = pd.DataFrame(df)
print(df_fr)
```

▸ 결과:

	country	year	GDP
0	US	2020	2.1
1	US	2021	2.2
2	US	2022	2.3
3	Korea	2021	10.2
4	Korea	2022	12.3
5	Korea	2023	15.1

◇ DataFrame을 이해할 때는 세 가지에 주목

▸ 열(column)은 ‘country‘, ‘year‘, ‘GDP‘와 같은 변수를, 행(row)은 각각의 관측치를 나타냄

- 가장 왼쪽의 ‘0, 1, 2, ...‘는 각 행을 구별하는 인덱스(index)

▸ 즉, 행 = 관측치

열 = 변수

index = 각 행의 위치 또는 이름

4.2.4 DataFrame의 기본 정보 확인

□ 데이터를 불러오거나 DataFrame을 만든 다음에는 먼저 데이터가 어떤 모습인지 확인해야 함

- 관측치가 수천 개 또는 수만 개인 실제 데이터에서는 전체 자료를 출력하기보다 head()와 tail()을 이용하여 데이터가 제대로 불러와졌는지를 먼저 확인하는 것이 편리

(1) 처음과 마지막 부분 확인

- 데이터의 처음 5개 행을 보여줌

```
df_fr.head()
```

- 마지막 5개 행을 보여줌

```
df_fr.tail()
```

(2) 데이터의 크기 확인

df_fr.shape

- 결과: (6, 3)
- 이는 DataFrame이 6개의 행과 3개의 열로 구성되어 있다는 의미

(3) 변수와 자료형 확인

- info()는 관측치의 수, 변수 이름, 각 변수의 자료형, 결측치 여부 등을 보여줌

df_fr.info()

(4) 기본적인 통계량 확인

- describe()는 수치형 변수의 평균, 표준편차, 최솟값, 최댓값, 사분위수 등 기본적인 기술통계량을 보여줌

df_fr.describe()

4.2.5 필요한 열 선택하기

□ DataFrame에서는 분석에 필요한 변수만 선택할 수 있음

▸ 앞의 fr에서 'GDP' 변수만 선택함

df_fr["GDP"]

▸ 결과는 하나의 Series가 됨

▸ 두 개 이상의 변수를 선택하려면 두 개의 변수 이름을 리스트로 전달

fr[["country", "GDP"]]

▸ 이 경우 결과는 DataFrame

▸ 대괄호의 개수에 주의

```
df_fr[“GDP“]                    # 하나의 열 → Series
df_fr[[“country“, “GDP“]]       # 여러 열 → DataFrame
```

□ 점 표기법

▸ 단점: 변수 이름에 공백이나 특수문자가 있는 경우에는 사용할 수 없음

```
df_fr.GDP
```

4.2.6 특정한 행과 열 선택하기: ‘loc‘와 ‘iloc‘

□ 두 방법의 차이:

- iloc → 위치(position)를 이용하여 선택
- loc → 이름(label)을 이용하여 선택

(1) iloc: 위치로 선택

- 첫 번째 행을 선택

```
df_fr.iloc[0]
```

- 특정한 행과 열을 함께 선택

```
df_fr.iloc[0, 2]
```

→ 첫 번째 행의 세 번째 열에 있는 값을 선택

(2) ‘loc‘: 이름으로 선택

- 인덱스가 0인 행에서 “GDP“ 열의 값을 선택

```
df_fr.loc[0, “GDP“]
```

4.2.7 전체 데이터 중 특정 조건을 만족하는 관측치만 선택해야 하는 경우

□ 예) 한국 자료만 선택하는 경우

df_fr[df_fr[“country“] == “Korea“]

▸ 결과:

```
   country   year    GDP
3    Korea   2021   10.2
4    Korea   2022   12.3
5    Korea   2023   15.1
```

‘‘‘

▸ 2022년 이후의 자료만 선택하는 경우:

df_fr[df_fr[“year“] >= 2022]

▸ 두 조건을 동시에 사용하는 경우

```
df_fr[ (df_fr[“country“] == “Korea“) & (df_fr[“year“] >= 2022) ]
```

▸ 이러한 방법은 앞에서 NumPy 배열을 이용해 배운 Boolean indexing과 같은 원리

4.2.8 데이터 정렬하기

□ GDP가 작은 국가·연도부터 정렬하려면 다음과 같이 작성

df_fr.sort_values(by="GDP")

- GDP가 큰 순서대로 정렬하려면

df_fr.sort_values(by="GDP", ascending=False)

- ascending=False는 내림차순으로 정렬한다는 의미

4.2.9 새로운 변수 만들기

□ DataFrame의 중요한 장점 중 하나는 기존 변수를 이용하여 새로운 변수를 쉽게 만들 수 있다는 것

□ 계산 방법을 이해하기 위한 간단한 예로 새로운 열을 추가

```
df_fr[“GDP_100“] = fr[“GDP“] * 100
```

▸ 그러면 기존 DataFrame에 GDP_100이라는 새로운 변수가 추가됨

4.2.10 결측치(Missing Value)

□ 경제 데이터에는 일부 값이 존재하지 않는 경우가 자주 있음 이러한 값을 결측치(missing value)라고 함

- Pandas에서는 결측치가 일반적으로 'NaN'으로 표시됨

□ 결측치가 있는지를 확인하려면:

```
df_fr.isna()
```

□ 각 변수에 결측치가 몇 개 있는지 확인하려면:

```
df_fr.isna().sum()
```

□ 결측치를 제거하려면:

```
df_fr.dropna()
```

□ 결측치를 특정 값(예: 0)으로 대체하려면:

```
df_fr.fillna(0)
```

◆ 다만 결측치를 무조건 삭제하거나 0으로 대체하는 것은 적절하지 않을 수 있음

▸ 어떻게 처리할 것인지는 결측치가 발생한 이유와 분석 목적을 고려하여 결정해야 함

4.2.11 데이터 결합: merge()

□ 실제 데이터 분석에서는 필요한 정보가 하나의 파일에 모두 들어 있지 않은 경우가 많음

□ 예) 한 데이터에는 국가별 GDP가 있고 다른 데이터에는 국가별 수출액이 있다고 하자.

① 국가별 GDP 자료: df_gdp

```
df_gdp = pd.DataFrame({
    "country": ["Germany", "Korea", "Japan", "UK"],
    "GDP": [10, 20, 30, 40]
})
```

② 국가별 수출액 자료: df_export

```
df_export = pd.DataFrame({
    "country": ["Germany", "Korea", "China", "France"],
    "export": [15, 25, 35, 45]
})
```

① 국가별 GDP 자료: df_gdp

	country	GDP
0	Germany	10
1	Korea	20
2	Japan	30
3	UK	40

② 국가별 수출액 자료: df_export

	country	export
0	Germany	15
1	Korea	25
2	China	35
3	France	45

◇ 두 데이터의 차이

국가	GDP 자료	수출 자료
Germany	✓	✓
Korea	✓	✓
Japan	✓	
UK	✓	
China		✓
France		✓

□ 두 데이터 공통으로 존재하는 변수 country를 기준으로 자료를 결합

▸ 이때 중요한 것은 어느 국가를 결합된 데이터에 남길 것인가?이며, 이것을 결정하는 것이 how

(1) Inner merge: 두 데이터에 모두 존재하는 관측치만 결합

▸ merge()에서 how를 지정하지 않으면 기본값은 “inner“

```
df_inner = pd.merge( df_gdp, df_export, on=“country“, how=“inner“ )
df_inner
```

▸ 실행 결과:

	country	GDP	export
0	Germany	10	15
1	Korea	20	25

▸ Germany와 Korea는 두 데이터에 모두 존재하기 때문에 남음.

반면 Japan과 UK는 수출 자료에 없고, China와 France는 GDP 자료에 없기 때문에 제외

(2) Left merge: 왼쪽 데이터를 기준으로 결합

- Left merge = 왼쪽 데이터의 관측치는 모두 유지하고, 오른쪽 데이터에서 일치하는 정보를 가져옴
- GDP 자료에 있는 국가를 모두 유지하는 경우

```
df_left = pd.merge( df_gdp, df_export, on="country", how="left" )
df_left
```

- 실행 결과:

	country	GDP	export
0	Germany	10	15
1	Korea	20	25
2	Japan	30	NaN
3	UK	40	NaN

- 왼쪽에 놓인 df_gdp의 Germany, Korea, Japan, UK가 모두 유지됨
- 그러나 Japan과 UK에 대응하는 수출 자료가 없으므로 export에는 NaN이 나타남

(3) Outer merge: 두 데이터에 존재하는 모든 관측치를 유지

▸ 어느 한쪽의 국가도 제외하지 않고 두 데이터에 존재하는 모든 국가를 유지

```
df_outer = pd.merge( df_gdp, df_export, on="country", how="outer" )
df_outer
```

▸ 실행 결과:

	country	GDP	export
0	China	NaN	35
1	France	NaN	45
2	Germany	10	15
3	Japan	30	NaN
4	Korea	20	25
5	UK	40	NaN

▸ 한쪽 데이터에만 존재하는 국가에서는 대응되는 값이 없으므로 NaN이 나타남

4.2.12 그룹별 통계량 계산: groupby()

□ 예) 국가별 GDP 평균을 계산하는 경우

▸ 데이터를 'country'별로 나눈 다음 → 각 국가의 'GDP' 평균을 계산

df.groupby("country")["GDP"].mean()

◆ groupby()의 원리: 세 단계

① Split : 데이터를 그룹으로 나눈다.

② Apply : 각 그룹에 평균, 합계 등의 계산을 적용한다.

③ Combine : 계산 결과를 다시 하나로 모은다.

```
# 국가별 GDP 평균

df.groupby("country")["GDP"].mean()


# 국가별 GDP 합계

df.groupby("country")["GDP"].sum()


# 국가와 연도별 GDP 평균

df.groupby(["country", "year"])[["GDP"]].mean()
```

4.2.13 Pandas의 데이터 전처리의 기본적인 과정: Python 코드와 연결

```
# ① 데이터 불러오기
df = pd.read_csv("data.csv")

# ② 데이터 확인
df.head()
df.info()

# ③ 필요한 데이터 선택
df[["country", "year", "GDP"]]

# ④ 결측치 확인
df.isna().sum()

# ⑤ 조건 맞는 데이터 선택
df[df["country"] == "Korea"]

# ⑥ 그룹별 분석
df.groupby("country")["GDP"].mean()
```

4.2.14 NumPy와 Pandas의 관계

자료구조	핵심적인 역할
Python List	여러 값을 묶어서 저장
NumPy Array	여러 숫자를 효율적으로 계산
Pandas Series	인덱스를 가진 1차원 데이터
Pandas DataFrame	행과 열로 구성된 표 형태의 데이터